c9r – Requirements & Design Documentation

Version 0.6 – 2026-02-22

1 Requirements Specification

1.1 Purpose

c9r (CanonFodder) is a reproducible pipeline that ingests music listening events (scrobbles), resolves artist-name ambiguity through record linkage and disambiguation, enriches data with metadata, and delivers interactive analytics. A trained binary classifier supports both automatic and user-reviewed canonisation of artist name variants.

1.2 Scope

In scope**:** automated data ingestion from **ListenBrainz** and last.fm; MusicBrainz enrichment; manual and ML-assisted canonisation via a CLI built on Click; Parquet-native storage with DuckDB as the analytical query engine; BI dashboards; Prefect workflow orchestration; model training and serving for artist-name record linkage; Docker packaging.

Out of scope**:** streaming playback, real-time recommendation engine, paid cloud hosting, Wikipedia-style disambiguation of distinct same-name artists (to be added in a later stage).

1.3 Problem Statement

The rapid growth of digitally available music has introduced systematic ambiguity in how data records map to artist entities. Records that appear to belong to distinct artists may, in fact, denote the same entity under variant spellings, transliterations, or stylisations. c9r addresses this by providing a record linkage pipeline whose core question is: to link or not to link.

The solution combines human input (gold-standard curation, established databases, user decisions) with programmatic and ML-based capabilities. A gold-standard table of labelled artist-name variant pairs is built, features are engineered from string similarity and character-level differences, and a binary classifier is trained to predict whether two name variants refer to the same entity.

1.4 Definitions & Acronyms

Term	Definition
AVC	artist_variants_canonized table
c9r / CF	CanonFodder project
CRISP-DM	Cross-Industry Standard Process for Data Mining
DWH	Data warehouse (Parquet file store queried via DuckDB)
LB	ListenBrainz
MB	MusicBrainz
Scrobble	One track-play event: (artist, track, timestamp)

UTS	Unix timestamp

1.5 Stakeholders

Priority	Stakeholder	Motivation
P0	Core developers/maintainers	Build and operate c9r
P0	Power users	Analyse personal listening history
P1	Casual users	Occasional insight into listening habits
P1	Musicology researchers	Reliable structured dataset
P2	Integration partners (LB, platforms)	Optional data exchange

1.6 Overall Description

c9r is a pull-based pipeline triggered manually or via Prefect. It converts raw scrobbles into a clean star schema stored as Parquet files, normalises artist aliases, enriches with metadata, and trains and serves a record-linkage classifier. Users interact through a Click CLI that exposes data gathering, canonisation review, model inference, and visual exploration.

1.7 Assumptions, Dependencies, Constraints

Python ≥ 3.12, PEP 8, 80 % test coverage target. External APIs: ListenBrainz (LB_TOKEN) and MusicBrainz (rate-limited at 1 req/s). Local filesystem read/write for Parquet cache. Optionally, a local MusicBrainz PostgreSQL mirror (via musicbrainz-docker) to bypass rate limits during bulk enrichment. Docker container run via: docker run c9r.

2 Software Requirements Document

2.1 System Interfaces

ID	Interface	Direction	Protocol / Lib	Description
I-01	LB API	Inbound	HTTPS REST	Fetch scrobbles via LBClient
I-02	MB API	Inbound	HTTPS REST	Artist lookup, search, aliases
I-03	File System	Bidirectional	Parquet (PyArrow)	Primary data store
I-04	DuckDB	Internal	duckdb-python	Analytical query engine over Parquet
I-05	CLI	Inbound	Click	User launches workflows
I-06	Prefect	Inbound	Python API	Scheduled orchestration
I-07	Model API	Internal	FastAPI/ASGI	Serve XGBoost classifier

2.2 Functional Requirements

ID	Description
FR-01	Fetch scrobbles – Pull recent tracks since last stored timestamp; persist to scrobble.parquet.
FR-02	Normalise scrobbles – Rename columns, convert UTS → UTC datetime, deduplicate.
FR-03	Bulk insert – Append normalised rows to Parquet file with deduplication guard.
FR-04	Artist enrichment – For new MBIDs, fetch country and aliases from MB; cache in artist_info.parquet.
FR-05	Canonisation – Group artist name variants, store mapping in AVC, apply to scrobble history.
FR-06	Gold standard – Build and curate labelled variant pairs via CLI review tool for model training.
FR-07	Model training – Train binary classifier (XGBoost) on gold standard with CRISP-DM phases.
FR-08	Model serving – Expose trained model via ASGI endpoint for auto-connect and user-review recommendations.
FR-09	BI frontend – Interactive dashboards using custom colour palettes via CLI menu.
FR-10	Retry & back-off – Network requests retry up to 8 times with exponential back-off.
FR-11	Workflow orchestration – One-click Prefect flow (cf_ingest) covering FR-01 through FR-09.
FR-12	Docker distribution – Build image with code, dependencies, model artefacts, and dashboards.

2.3 Performance Requirements

Ingest 10 000 scrobbles per minute on consumer-grade hardware. End-to-end Prefect flow completes within 15 minutes for 1 million scrobbles. Dashboard renders top-10 artists within 2 seconds via DuckDB over Parquet.

2.4 Data Requirements

All timestamps stored in UTC with timezone awareness. Parquet files partitioned by year for the scrobble table. Primary key semantics enforced at the application layer (unique composite of artist_name + track_title + play_time). Parquet compression: zstd.

2.5 Reliability & Availability

Retry logic on HTTP 5xx and network errors (FR-10). Schema changes tracked via versioned Parquet schemas with migration scripts. Unit and integration tests; CI fails if coverage < 80 % or linting errors.

2.6 Security & Privacy

Secrets stored in .env; mounted via Docker secrets in production. Only read-only LB scope. GDPR compliance: user can purge all personal data via CLI command.

2.7 Acceptance Criteria

All functional requirements demonstrably fulfilled. Ingest 300k scrobbles without duplication. Dashboard renders top-10 artists within 2 seconds. Classifier exceeds a threshold on F_1 score (to be determined during modelling), with emphasis on precision to minimise false merges. Canonised top-artist ranking matches a manually verified ground truth.

3 High-Level Design

3.1 Architectural Overview

The architecture follows a medallion-style pattern with three layers. The source layer fetches raw data from ListenBrainz and MusicBrainz APIs via resilient HTTP clients. The ELT layer normalises, deduplicates, enriches, and canonises data, writing results as Parquet files. The analytical layer uses DuckDB to query Parquet files directly for dashboards and reporting. A model-serving sidecar exposes the trained XGBoost classifier via an ASGI endpoint.

3.2 Key Architectural Decisions

Parquet + DuckDB replaces MySQL. The previous iteration used MySQL as the primary store, which introduced server management overhead and was a bottleneck for analytical queries. Parquet is columnar, compresses well with zstd, preserves types, and supports predicate pushdown. DuckDB queries Parquet files natively, runs in-process (no server), and is orders of magnitude faster than row-oriented engines for aggregations. For c9r's single-user, batch-oriented, read-heavy workload this is an ideal fit. Transactional guarantees (deduplication, upserts) are handled at the application layer via Pandas before writing Parquet.

Click replaces windows-curses CLI. The previous curses-based interface was fragile across platforms and painful to extend. Click provides composable command groups, automatic help generation, parameter validation, and works identically on Linux, macOS, and Windows. Interactive prompts are handled via Click's built-in prompt and confirmation utilities.

Prefect retained for orchestration. Prefect 3.0 fits c9r's batch-oriented, single-user workflow model. Kafka's strength is high-throughput, real-time event streaming across distributed consumers – capabilities that are overkill for a pipeline that runs periodically and processes batches of scrobbles. Kafka should be re-evaluated only if c9r moves to a multi-user, real-time streaming architecture in the future.

Optional local MusicBrainz mirror. The MB API rate limit (1 req/s) makes bulk enrichment painfully slow. A local PostgreSQL mirror (via musicbrainz-docker) provides unlimited query throughput for enrichment. This is the only justified use of a relational DB in the architecture.

3.3 Key Components

Component	Responsibilities
Source layer (HTTP/)	Fetch scrobbles (LB), fetch artist data (MB), resilient client with retries
ELT layer (corefunc/)	Normalisation, canonisation, enrichment, gold-standard curation, Parquet I/O
Persistence (PQ/)	Parquet file store: scrobble, artist_info, avc, country codes, user_country
Query engine	DuckDB in-process – SQL over Parquet for dashboards and ad-hoc analytics
ML pipeline (canon.py)	Feature engineering, XGBoost training, evaluation, model persistence

Model server	ASGI endpoint serving XGBoost predictions for canonisation recommendations
Orchestration	Prefect flows and Click CLI commands
BI frontend	CLI menu and dashboards using palettes.json

3.4 Data Flow

1. Prefect triggers the fetch-and-ingest task.
2. LBClient/lfAPI pulls new scrobbles, returns JSON.
3. ELT layer converts to DataFrame, deduplicates, appends to scrobble.parquet.
4. MB enrichment fetches missing artist rows; upserts into artist_info.parquet.
5. Canonisation (CLI or model) groups variants, updates avc.parquet.
6. User opens dashboard; DuckDB queries Parquet files on the fly.

3.5 Data Schema

Parquet File	Key Columns	Constraints (app-layer)
scrobble.parquet	artist_name, track_title, play_time, artist_mbid, album_title	Unique: (artist_name, track_title, play_time). Partitioned by year.
artist_info.parquet	artist_name, mbid, country, disambiguation_comment, aliases	Unique: mbid
avc.parquet	artist_variants_hash, canonical_name, to_link, comment, stamp	PK: artist_variants_hash
c.parquet	ISO-2, ISO-3, en_name, hu_name	PK: ISO-2
uc.parquet	country_code, start_date, end_date	Period check on dates

3.6 Technology Stack

Layer	Technology	Rationale
Language	Python 3.12+	Modern typing, datetime.UTC, broad ecosystem
Storage	Parquet (PyArrow, zstd)	Columnar, portable, efficient compression
Query engine	DuckDB	In-process OLAP; native Parquet; no server needed
Orchestration	Prefect 3.0	Python-native, simple local dev, no server dependency for basic use
CLI	Click	Composable commands, cross-platform, auto-generated help
ML	XGBoost + scikit-learn	Strong tree-based classification; lightweight deployment
Model serving	FastAPI (ASGI)	Async, lightweight, auto-generated OpenAPI docs
Containerisation	Docker	Reproducible deployment
Optional RDBMS	PostgreSQL (local MB mirror)	Bypass MB API rate limit for bulk enrichment

4 Low-Level Design

4.1 Module Breakdown

Module	Key Functions / Classes	Notes
lblink.py	LBClient(), get_listens()	Pagination + auth; respects rate limit; yields DataFrames
helpers/io.py	append_to_parquet(), dump_parquet()	Dedup-aware Parquet I/O with PyArrow
mbAPI.py	lookup_mb_for(), fetch_country()	Tenacity retry; 1 req/s; local MB fallback
helpers/cli.py	unify_artist_names_cli()	Click prompts for manual grouping; hashes variants
canon.py	train_model(), evaluate()	Feature engineering, XGBoost pipeline, model persistence
model_server.py	predict(), health_check()	FastAPI ASGI serving XGBoost from xgb.json
dataprofiler.py	quick_viz()	Charts with palette from palettes.json
helpers/stats.py	length_stats(), show_misclassified()	Feature engineering helpers, evaluation utilities

4.2 Core Data Model

Primary fact record (scrobble):

```
    artist_name: str(350)     album_title: str(350)     track_title: str(350)
artist_mbid: str(36) | None     play_time: datetime(timezone=True)
```

4.3 Algorithms

Canonisation: RapidFuzz string similarity + DBSCAN clustering + manual anchors + XGBoost classifier.

Feature engineering: Six pairwise RapidFuzz scores, string-length statistics (signature length, average name length), character-level difference features.

Bulk insert: Chunks of 10 000 rows with application-layer deduplication before Parquet append.

4.4 Error Handling & Logging

HTTP client retries ×8 with exponential back-off; logs to stdout. Non-fatal errors escalate via raise_for_status in ingest context. Parquet schema mismatches abort with clear error message.

4.5 Configuration

.env parsed by python-dotenv; required keys validated at startup. Default fallback uses local Parquet files with no external dependencies for tests.

4.6 Testing Strategy

pytest suite in tests/ with fixtures. CI matrix: unit, integration with temporary Parquet files, coverage gate ≥ 80 %.

5 Traceability Matrix

RS Section	FR IDs
§1.2 Scope – ingest, enrich, store	FR-01 … FR-05
§1.3 Problem statement – record linkage	FR-06, FR-07, FR-08
§1.2 Scope – BI frontend	FR-09
§1.7 Constraints – network resilience	FR-10
§1.6 Description – orchestration	FR-11
§1.7 Constraints – Docker	FR-12

6 Open Issues / Future Work

OAuth flow for ListenBrainz. Real-time streaming ingest (re-evaluate Kafka at that point). Web-based dashboard with palette auto-theming. Expanded gold standard for multilingual artist names. Experiment with ensemble approaches and alternative classifiers.